Ghee's design is elegant, but it uses so much metaprogramming that it might confuse you at first. Let's wrap it up in just two points:

- Ghee stores GitHub objects as dynamic hashes. You can access the attributes of these hashes by calling their *Ghost Methods (57)*, such as `url` and `description`.
- Ghee also wraps these hashes inside proxy objects that enrich them with additional methods. A proxy does two things. First, it implements methods that require specific code, such as `star`. Second, it forwards methods that just read data, such as `url`, to the wrapped hash.

Thanks to this two-level design, Ghee manages to keep its code very compact. It doesn't need to define methods that just read data, because those methods are Ghost Methods. Instead, it can just define the methods that need specific code, like `star`.

This dynamic approach also has another advantage: Ghee can adapt automatically to some changes in the GitHub APIs. For example, if GitHub added a new field to gists (say, `lines_count`), Ghee would support calls to `Ghee::API::Gists#lines_count` without any changes to its source code, because `lines_count` is just a Ghost Method—actually a chain of *two* Ghost Methods.

Spell: Dynamic Proxy

An object such as `Ghee::ResourceProxy`, which catches Ghost Methods and forwards them to another object, is called a *Dynamic Proxy*.

Refactoring the Computer Class (Again)

"Okay, you now know about `method_missing`," Bill says. "Let's go back to the `Computer` class and remove the duplication."

Once again, here's the original `Computer` class:

methods/computer/duplicated.rb

```
class Computer
  def initialize(computer_id, data_source)
    @id = computer_id
    @data_source = data_source
  end

  def mouse
    info = @data_source.get_mouse_info(@id)
    price = @data_source.get_mouse_price(@id)
    result = "Mouse: #{info} ($#{price})"
    return "* #{result}" if price >= 100
    result
  end
```